



ФГБОУ ВО «Петрозаводский государственный университет»

Институт математики и информационных технологий

Учебная дисциплина «Языки программирования и методы трансляции»

Задача 4. Разбор стадий трансляции логических выражений в псевдомашинный код

Исследование провёл студент группы 22407 Гордеев Никита

Дата выполнения работы 05.04.2025 (Версия 3)

История изменений

Версия 1 (22.03.2025): Разобрана простая программа с однобуквенными переменными и базовой условной конструкцией (И, ИЛИ)

Версия 2 (30.03.2025): Выбрана более сложная программа с осмысленными именами переменных, вложенными условиями (И, ИЛИ, ОТРИЦАНИЕ) и расширенным описанием БНФ. Добавлены комментарии БНФ, поясняющие логику.

Версия 3 (05.04.2025): Разобраны стадии трансляции логических выражений в псевдо машинный код. Предложены варианты оптимизации.

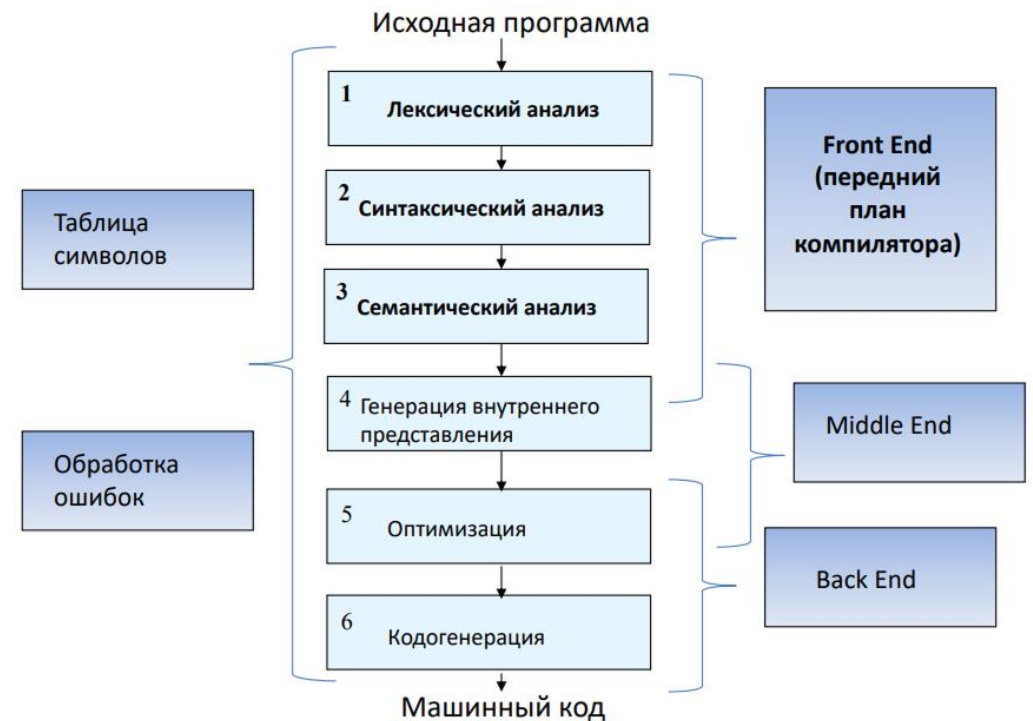
Версия 4 (06.05.2025): Добавлен порядок обхода дерева, проведена оптимизация по числу регистров и команд. В примечаниях слайдов добавлена расшифровка сокращений терминов.

.

Формулировка задачи

1. Выбрать программу содержащую логические выражения.
2. Предложить набор БНФ¹.
3. Построить синтаксическое дерево.
4. Разобрать стадии трансляции логических выражений в псевдомашинный код.
5. После выполнения стадии генерации кода предложить варианты оптимизации (по числу регистров, по числу команд).

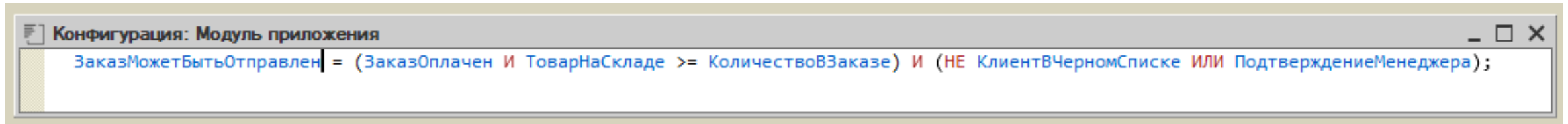
Фазы компиляции



¹ БНФ (форма Бэкуса — Наура) — это формальная система описания синтаксиса, в которой одни синтаксические категории последовательно определяются через другие категории.

Выбор языка программирования и программы

- Язык 1С — это встроенный язык платформы «1С: Предприятие».
- Разбираемая программа проверяет возможность отгрузки товара по заказу, анализируя три ключевых условия: наличие полной оплаты, достаточное количество товара на складе, а также отсутствие клиента в черном списке или наличие подтверждения менеджера.



The screenshot shows a window titled "Конфигурация: Модуль приложения". Inside, a logical formula is entered in a text field:

```
ЗаказМожетБытьОтправлен = (ЗаказОплачен И ТоварНаСкладе >= КоличествоВЗаказе) И (НЕ КлиентВЧерномСписке ИЛИ ПодтверждениеМенеджера);
```

- Ошибки в таких расчетах могут привести к финансовым потерям, что делает их обработку критически важной.

Лексический анализатор

	Имя	Вид	Значение
1	ЗаказМожетБытьОтправлен	Идентификатор	Булева переменная
2	=	Оператор присваивания	Булева переменная
3	(	Скобки	
4	ЗаказОплачен	Идентификатор	Булева переменная
5	И	Логический оператор	
6	ТоварНаСкладе	Идентификатор	Числовая переменная
7	>=	Оператор сравнения	
8	КоличествоВЗаказе	Идентификатор	Числовая переменная
9	)	Скобки	
10	И	Логический оператор	
11	(	Скобки	
12	НЕ	Логический оператор	
13	КлиентВЧерномСписке	Идентификатор	Булева переменная
14	ИЛИ	Логический оператор	
15	ПодтверждениеМенеджера	Идентификатор	Булева переменная
16	)	Скобки	
17	;	Спецсимвол	

Синтаксический анализатор

<Программа> ::= <Присваивание> ";" // Присваивания с обязательной точкой с запятой

<Присваивание> ::= <Идентификатор> "=" <ЛогическоеВыражение> // Присваивание значения переменной

<ЛогическоеВыражение> ::= <ЛогическоеУмножение> ("ИЛИ" <ЛогическоеУмножение>)* // Логическое сложение (ИЛИ) с
возможностью продолжения

<ЛогическоеУмножение> ::= <ЛогическоеОтрицание> ("И" <ЛогическоеОтрицание>)* // Логическое умножение (И) с
возможностью продолжения

<ЛогическоеОтрицание> ::= ("НЕ") <ЛогическийТерм> // либо просто терм, либо с оператором НЕ

<ЛогическийТерм> ::= <Сравнение> | "(" <ЛогическоеВыражение> ")" | <Идентификатор> // минимальная неделимая часть
логического выражения

<Сравнение> ::= <Выражение> <ОператорСравнения> <Выражение> // Сравнение двух выражений

<ОператорСравнения> ::= ">=" | "<=" | "==" | "!=" | ">" | "<" // Операторы сравнения

<Идентификатор> ::= <Буква> (<Буква> | <Цифра> | "_")* // Начинается с буквы, может содержать буквы, цифры и подчеркивания

<Число> ::= <Целое> | <Вещественное> // Числовые литералы

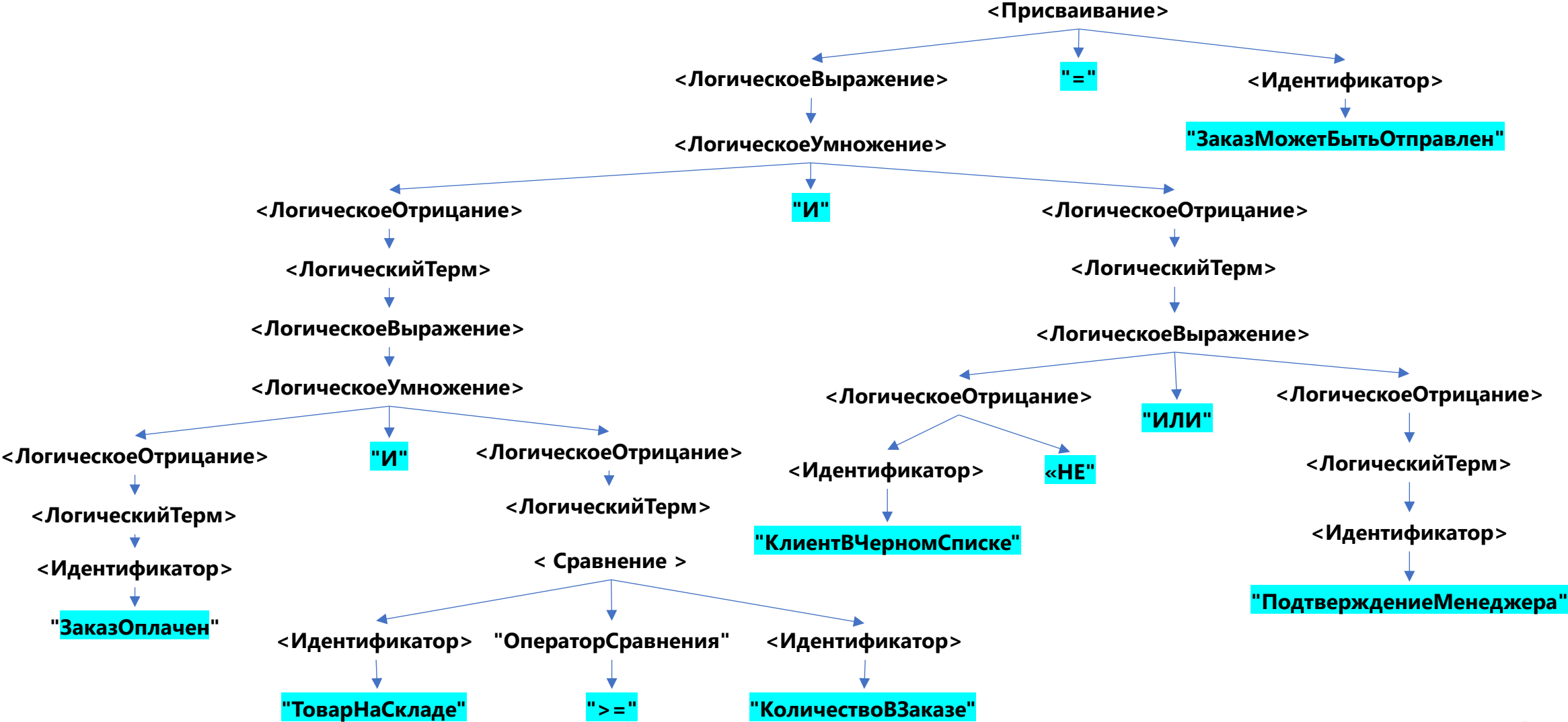
<Целое> ::= <Цифра>+ // Последовательность цифр

<Вещественное> ::= <Цифра>+ "." <Цифра>+ // Число с плавающей точкой

<Буква> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z" | "а" | "б" | ... | "я" | "А" | "Б" | ... | "Я" // Латиница и кириллица (строчные и заглавные)

<Цифра> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Синтаксическое дерево



! В дерево не влезли открывающие "(" и закрывающие ")" скобки, а также точки с запятой ";", поэтому они явно не отображены. Однако следует иметь в виду, что они присутствуют в полной структуре кода.

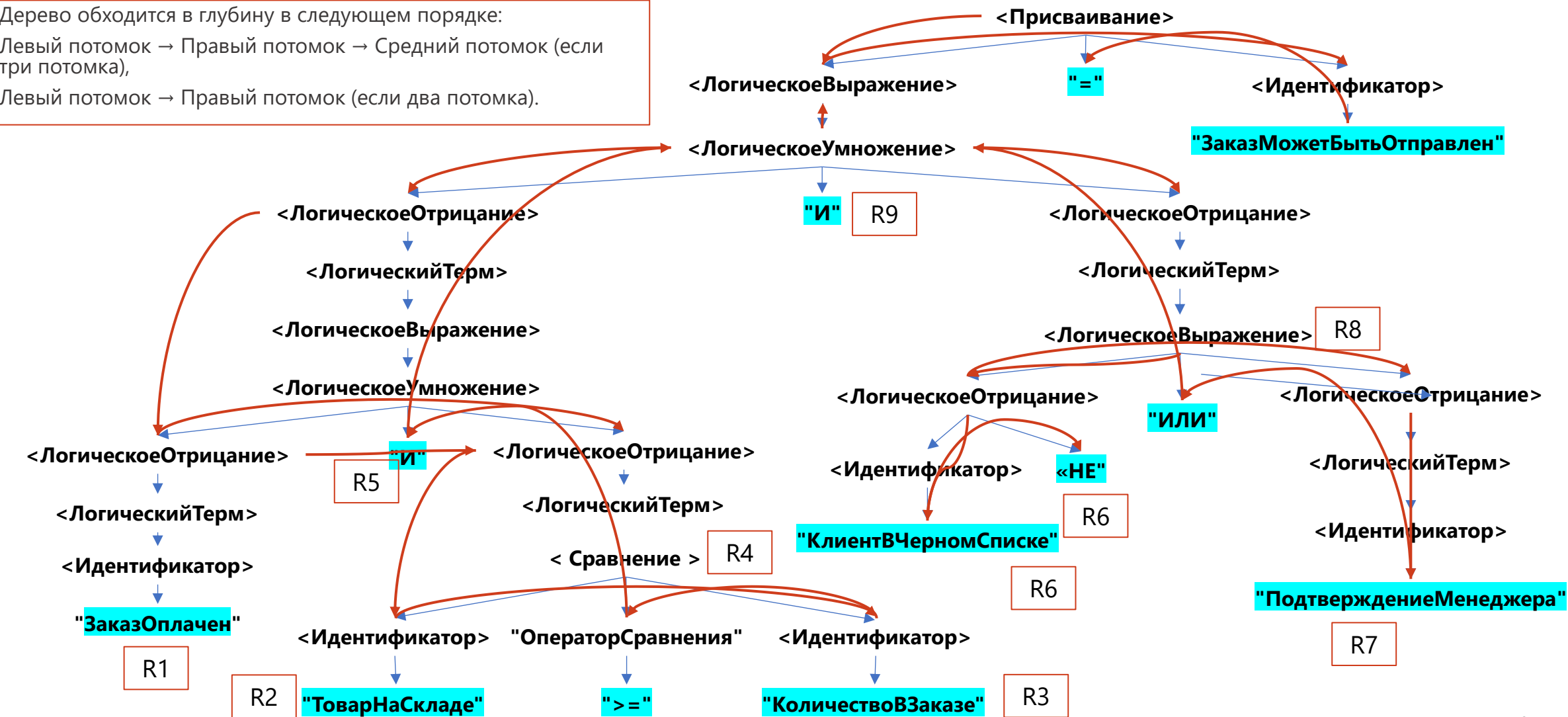
Семантический анализатор

Синтаксис программы представляет собой псевдо машинный язык, который использует набор команд для выполнения операций над значениями в регистрах. Регистр - это небольшой блок памяти, в котором хранится значение. Основные команды языка:

- | | |
|---------------------------------------|---|
| 1. LOAD значение, регистр | // загрузка значения в регистр |
| 2. STORE регистр, значение | // сохранение значения из регистра в память |
| 3. AND операнд1, операнд2, результат | // логическое И |
| 4. OR операнд1, операнд2, результат | // логическое ИЛИ |
| 5. CMP операнд1, операнд2 | // сравнение |
| 6. JMP метка | // условный переход |
| 7. JE метка | // переход если равно |
| 8. JNE метка | // переход если не равно |
| 9. JG метка | // переход если больше |
| 10. JL метка | // переход если меньше |
| 11. JGE метка | // переход если больше или равно |
| 12. JLE метка | // переход если меньше или равно |
| 13. MUL операнд1, операнд2, результат | // умножение |
| 14. ADD операнд1, операнд2, результат | // сложение |
| 15. SUB операнд1, операнд2, результат | // вычитание |
| 16. DIV операнд1, операнд2, результат | // деление |
| 17. MOV источник, приемник | // перемещение значения между регистрами |
| 18. PUSH значение | // поместить значение в стек |
| 19. POP регистр | // извлечь значение из стека в регистр |
| 20. CALL метка | // вызов подпрограммы |
| 21. RET | // возврат из подпрограммы |

Синтаксическое дерево

Дерево обходится в глубину в следующем порядке:
Левый потомок → Правый потомок → Средний потомок (если три потомка),
Левый потомок → Правый потомок (если два потомка).



! В дерево не влезли открывающие "(" и закрывающие ")" скобки, а также точки с запятой ";", поэтому они явно не отображены. Однако следует иметь в виду, что они присутствуют в полной структуре кода.

Генерация внутреннего представления

// Семантический анализ условия "ЗаказМожетБытьОтправлен"

// ЗаказМожетБытьОтправлен = (ЗаказОплачен И ТоварНаСкладе >= КоличествоВЗаказе) И (НЕ КлиентВЧерномСписке ИЛИ ПодтверждениеМенеджера);

// Первая часть: проверка наличия товара и оплаты:

1. LOAD ЗаказОплачен, R1 // Загружаем статус оплаты в регистр R1, (Значение 1 - заказ оплачен, 0 - не оплачен)

2. LOAD ТоварНаСкладе, R2 // Загружаем количество товара на складе в R2

3. LOAD КоличествоВЗаказе, R3 // Загружаем требуемое количество в R3

4. CMP R2, R3 // Проверяем, достаточно ли товара на складе для выполнения заказа - сравниваем ТоварНаСкладе и КоличествоВЗаказе

5. JGE ДостаточноТовара // Если на складе >= чем в заказе, переходим на метку ДостаточноТовара

6. MOV 0, R4 // Иначе сохраняем 0 (False) в R4

7. JMP ПроверкаОплаты // Переходим к проверке оплаты

8. ДостаточноТовара:

9. MOV 1, R4 // Сохраняем 1 (True) в R4

10. ПроверкаОплаты:

11. AND R1, R4, R5 // R5 = ЗаказОплачен И (ТоварНаСкладе >= КоличествоВЗаказе)

// Вторая часть: проверка клиента и подтверждения

12. LOAD КлиентВЧерномСписке, R6 // Загружаем флаг нахождения клиента в черном списке (1 - в списке, 0 - нет)

13. NOT R6, R6 // Инвертируем значение (НЕ КлиентВЧерномСписке)

14. LOAD ПодтверждениеМенеджера, R7 // Загружаем флаг подтверждения менеджера (1 - подтверждено, 0 - нет)

15. OR R6, R7, R8 // R8 = (НЕ КлиентВЧерномСписке) ИЛИ ПодтверждениеМенеджера

// Сохраняем итоговый результат (1 - заказ можно отправить, 0 - нельзя)

16. AND R5, R8, R9 // R9 = (R5) И (R8) - итоговый результат

17. STORE R9, ЗаказМожетБытьОтправлен // Сохраняем результат

Оптимизация по числу регистров

// Оптимизированная версия с использованием всего 3 регистров (вместо 9)
// ЗаказМожетБытьОтправлен = (ЗаказОплачен И ТоварНаСкладе >= КоличествоВЗаказе) И (НЕ КлиентВЧерномСписке ИЛИ ПодтверждениеМенеджера);

// Первая часть: проверка наличия товара и оплаты:

1. LOAD ТоварНаСкладе, R1 // Загружаем количество товара на складе в R1
2. LOAD КоличествоВЗаказе, R2 // Загружаем требуемое количество в R2

3. CMP R1, R2 // Проверяем, достаточно ли товара на складе для выполнения заказа - сравниваем ТоварНаСкладе и КоличествоВЗаказе
4. JGE ДостаточноТовара // Если на складе >= чем в заказе, переходим на метку ДостаточноТовара
5. MOV 0, R1 // Иначе сохраняем 0 (False) в R1. Переиспользуем R1 (флаг достаточности товара)
6. JMP ПроверкаОплаты // Переходим к проверке оплаты

7. ДостаточноТовара:
8. MOV 1, R1 // Сохраняем 1 (True) в R1. Переиспользуем R1 (флаг достаточности товара)

9. ПроверкаОплаты:
10. LOAD ЗаказОплачен, R2 // Загружаем статус оплаты в регистр R2, (Значение 1 - заказ оплачен, 0 - не оплачен), переиспользуем R2
11. AND R2, R1, R1 // R1 = ЗаказОплачен И (ТоварНаСкладе >= КоличествоВЗаказе)

// Вторая часть: проверка клиента и подтверждения

12. LOAD КлиентВЧерномСписке, R2 // Загружаем флаг нахождения клиента в черном списке (1 - в списке, 0 - нет). Переиспользуем R2
13. NOT R2, R2 // Инвертируем значение (НЕ КлиентВЧерномСписке)
14. LOAD ПодтверждениеМенеджера, R3 // Загружаем флаг подтверждения менеджера (1 - подтверждено, 0 - нет)

15. OR R2, R3, R2 // R2 = (НЕ КлиентВЧерномСписке) ИЛИ ПодтверждениеМенеджера

// Сохраняем итоговый результат (1 - заказ можно отправить, 0 - нельзя)

16. AND R1, R2, R1 // R1 = (R5) И (R8) - итоговый результат
17. STORE R1, ЗаказМожетБытьОтправлен // Сохраняем результат

Оптимизация по числу команд

// Оптимизированная версия с использованием всего 16 команд (вместо 17)

// ЗаказМожетБытьОтправлен = (ЗаказОплачен И ТоварНаСкладе >= КоличествоВЗаказе) И (НЕ КлиентВЧерномСписке ИЛИ ПодтверждениеМенеджера);

// Первая часть: проверка наличия товара и оплаты:

1. LOAD ЗаказОплачен, R1 // Загружаем статус оплаты в регистр R1, (Значение 1 - заказ оплачен, 0 - не оплачен)

2. LOAD ТоварНаСкладе, R2 // Загружаем количество товара на складе в R2

3. LOAD КоличествоВЗаказе, R3 // Загружаем требуемое количество в R3

4. CMP R2, R3 // Проверяем, достаточно ли товара на складе для выполнения заказа - сравниваем ТоварНаСкладе и КоличествоВЗаказе

5. JL НедостаточноТовара // Если товара на складе меньше, чем заказано, переходим на метку НедостаточноТовара

6. AND R1, 1, R5 // Если товара достаточно, проверяем, оплачен ли заказ (1 AND ЗаказОплачен). Результат сохраняем в R5.

7. JMP ПроверкаКлиента // Переходим к проверке данных клиента

8. НедостаточноТовара: // Метка, если товара недостаточно

9. MOV 0, R5 // Сохраняем 0 (False) в R5 (заказ не может быть отправлен из-за недостатка товара)

// Вторая часть: проверка клиента и подтверждения

10. ПроверкаКлиента: // Начало блока проверки клиента

11. LOAD КлиентВЧерномСписке, R6 // Загружаем флаг нахождения клиента в черном списке (1 - в списке, 0 - нет)

12. NOT R6, R6 // Инвертируем значение: теперь 0 - в списке, 1 - не в списке

13. LOAD ПодтверждениеМенеджера, R7 // Загружаем флаг подтверждения менеджера (1 - подтверждено, 0 - нет)

14. OR R6, R7, R8 // Объединяем условия R8 = (НЕ КлиентВЧерномСписке) ИЛИ ПодтверждениеМенеджера

// Сохраняем итоговый результат (1 - заказ можно отправить, 0 - нельзя)

15. AND R5, R8, R9 // R9 = (R5) И (R8) - итоговый результат

16. STORE R9, ЗаказМожетБытьОтправлен // Сохраняем результат

Оптимизация по числу команд и регистров

// Оптимизированная версия с использованием всего 16 команд (вместо 17), 3 регистров (вместо 9)
// ЗаказМожетБытьОтправлен = (ЗаказОплачен И ТоварНаСкладе >= КоличествоВЗаказе) И (НЕ КлиентВЧерномСписке ИЛИ ПодтверждениеМенеджера);

// Первая часть: проверка наличия товара и оплаты:

1. LOAD ЗаказОплачен, R1 // Загружаем статус оплаты в регистр R1, (Значение 1 - заказ оплачен, 0 - не оплачен)
2. LOAD ТоварНаСкладе, R2 // Загружаем количество товара на складе в R2
3. LOAD КоличествоВЗаказе, R3 // Загружаем требуемое количество в R3

4. CMP R2, R3 // Проверяем, достаточно ли товара на складе для выполнения заказа - сравниваем ТоварНаСкладе и КоличествоВЗаказе
5. JL НедостаточноТовара // Если товара на складе меньше, чем заказано, переходим на метку НедостаточноТовара
6. AND R1, 1, R1 // Если товара достаточно, проверяем, оплачен ли заказ (1 AND ЗаказОплачен). Результат сохраняем в R1.
7. JMP ПроверкаКлиента // Переходим к проверке данных клиента

8. НедостаточноТовара: // Метка, если товара недостаточно
9. MOV 0, R1 // Сохраняем 0 (False) в R1 (заказ не может быть отправлен из-за недостатка товара)

// Вторая часть: проверка клиента и подтверждения

10. ПроверкаКлиента: // Начало блока проверки клиента
11. LOAD КлиентВЧерномСписке, R2 // Загружаем флаг нахождения клиента в черном списке (1 - в списке, 0 - нет)
12. NOT R2, R2 // Инвертируем значение: теперь 0 - в списке, 1 - не в списке
13. LOAD ПодтверждениеМенеджера, R3 // Загружаем флаг подтверждения менеджера (1 - подтверждено, 0 - нет)
14. OR R2, R3, R2 // Объединяем условия R2 = (НЕ КлиентВЧерномСписке) ИЛИ ПодтверждениеМенеджера

// Сохраняем итоговый результат (1 - заказ можно отправить, 0 - нельзя)

15. AND R1, R2, R1 // R1 = (R1) И (R2) - итоговый результат
16. STORE R1, ЗаказМожетБытьОтправлен // Сохраняем результат

Источники

- Корзун Д. Ж. Практикум по формальным грамматикам и языкам. Учебное пос. Петрозаводск: Изд-во ПетрГУ, 2004. 46 с. Электронная версия.
- Учебник Новосибирского государственного технического университета (автор Малявко А.А.) «Формальные языки и компиляторы», 2014 год, 431 стр., ISBN: 978-5-7782-2318-9.